
DOCUMENTACION TECNICA

Manual Tecnico

Pastas Orlando

Documentacion tecnica del sistema ERP + E-commerce +
Landing page. Arquitectura, modelo de datos, API endpoints,
seguridad y guia de despliegue.

MARZO 2025

Next.js 16 · TypeScript · Prisma · Turso

Indice

- 1. Resumen del proyecto**
- 2. Arquitectura del sistema**
- 3. Estructura de carpetas**
- 4. Modelo de datos (Prisma)**
- 5. Modulos principales y API endpoints**
- 6. Variables de entorno (.env)**
- 7. Despliegue**
- 8. Seguridad**
- 9. Dependencias principales**
- 10. Guia de instalacion local**

1. Resumen del proyecto

Campo	Detalle
Nombre	Pastas Orlando
Tipo	ERP + E-commerce + Landing page
Tecnologia principal	Next.js 16 + TypeScript + Tailwind CSS 4 + Prisma + Turso (libSQL)
Despliegue	Vercel (produccion), SQLite (desarrollo local)
Dominio	https://laspastasdeorlando.com.ar
Repositorio	https://github.com/orlandocandia/pastas-orlando

Pastas Orlando es un sistema integral que combina un ERP para la gestion interna de la empresa de pastas, una plataforma de e-commerce para la venta online, y una landing page publica para la captacion de clientes. El sistema esta construido con tecnologias modernas y desplegado en la nube, ofreciendo una solucion completa para la administracion de produccion, inventario, ventas, compras, logistica y seguridad.

2. Arquitectura del sistema

El sistema sigue una arquitectura moderna basada en Next.js con App Router, donde el frontend y backend coexisten en una sola aplicacion. A continuacion se detallan las tecnologias utilizadas en cada capa:

Capa	Tecnologia
Frontend	Next.js 16 (App Router), React 19 Server Components, Tailwind CSS 4, shadcn/ui
Backend	Next.js API Routes (60+ endpoints)
Base de datos	Turso (libSQL) en produccion / SQLite en desarrollo, via Prisma ORM
Autenticacion	NextAuth.js v4 (Credentials provider + 2FA con Speakeasy TOTP)
Notificaciones	Nodemailer (SMTP pooled) + WhatsApp (TextMeBot API)
Mapas	Leaflet + OpenStreetMap
Reportes	SheetJS (Excel/XLSX), @react-pdf/renderer (PDF), jsPDF
Subida de imagenes	Vercel Blob Storage + fallback local
Estado	Zustand (cliente) + TanStack React Query (servidor)
Formularios	React Hook Form + Zod validation

La arquitectura se beneficia de los React Server Components para renderizado del lado del servidor, reduciendo el JavaScript enviado al cliente. Las API Routes actuan como un backend RESTful completo, conectandose a la base de datos Turso mediante Prisma ORM que abstrae las diferencias entre SQLite (desarrollo local) y libSQL (produccion).

3. Estructura de carpetas

El proyecto sigue la estructura estandar de Next.js con App Router, organizando los componentes por funcionalidad y manteniendo una separacion clara entre logica de negocio y presentacion:

```
src/
+-- app/                                # Rutas (App Router) + API endpoints
|   +-- (dashboard)/admin/             # Dashboard privado
|   +-- (auth)/                        # Paginas de autenticacion
|   +-- api/                           # 60+ API endpoints
+-- components/
|   +-- sections/                      # Secciones de la landing page
|   +-- admin/                        # Componentes del dashboard
|   +-- ui/                           # Componentes shadcn/ui (40+)
|   +-- layout/                       # Navbar, Footer, ScrollToTop
|   +-- icons/                        # Iconos custom (WhatsApp, MercadoPago)
|   +-- opiniones/                    # Formulario y carrusel de opiniones
|   +-- logistica/                    # Mapas Leaflet
|   +-- print/                        # Vistas de impresion (etiquetas, hoja ruta)
+-- hooks/                            # Custom React hooks
+-- lib/                              # Utilidades y servicios compartidos
|   +-- db.ts                         # Prisma client (auto-detecta SQLite/Turso)
|   +-- auth-helpers.ts               # requireAuth(), requireRole(), requireAdmin()
|   +-- permisos-service.ts           # Verificacion de permisos por modulo
|   +-- auditoria-service.ts          # Registro de auditoria
|   +-- email.ts                      # Envio de emails HTML (password reset)
|   +-- smtp-transporter.ts           # Transporter SMTP pooled (pool: true)
|   +-- whatsapp-admin.ts             # WhatsApp via TextMeBot
|   +-- upload.ts                     # Subida de imagenes (Vercel Blob + local)
|   +-- notificaciones-service.ts     # Notificaciones + alertas automaticas
|   +-- utils.ts                      # cn() class merge
+-- instrumentation.ts                # Setup de variables Turso
prisma/
+-- schema.prisma                     # 40 modelos
+-- seed.ts / seed-completo.ts
public/
+-- images/                           # Assets estaticos
+-- logo.svg
```

La carpeta **app/** contiene las rutas del App Router de Next.js, organizadas en route groups como (dashboard) y (auth) para layouts compartidos. La carpeta **components/** separa los componentes por dominio funcional, mientras que **lib/** centraliza la logica de negocio y servicios compartidos.

4. Modelo de datos (Prisma)

El esquema de Prisma define 40 modelos organizados por modulo funcional. Esta estructura refleja la complejidad del sistema ERP y permite una gestion completa del ciclo de negocio de la empresa:

Modulo	Modelos
Landing page	Producto, Opinion, InteraccionWhatsApp, Consulta
Geografia	Pais, Provincia, Departamento, Municipio
Personas	Persona, Contacto, TipoPersona, TipoContacto, Direccion, TipoDireccion
Usuarios	Usuario, Rol, UsuarioRol, Permiso, RolPermiso, Sesion
Inventario	UnidadMedida, MateriaPrima, CategoriaMateriaPrima, Insumo, TipoInsumo, Marca, ProductoTerminado, CategoriaProductoTerminado
Recetas/Produccion	Receta, DetalleReceta, Produccion, DetalleProduccionConsumo, DetalleProduccionGenerado
Comercio	Compra, DetalleCompra, PedidoProveedor, DetallePedidoProveedor, PedidoCliente, DetallePedidoCliente, Venta, DetalleVenta, Presupuesto, DetallePresupuesto, ReservaCliente, FormaPago, EstadoGeneral
Stock/Auditoria	StockMovement, Auditoria
Seguridad	Usuario2FA, LogAcceso, SesionActiva
Logistica	PuntoEncuentro, Entrega, NotificacionEntrega
Notificaciones	PlantillaNotificacion, Notificacion, AlertaConfiguracion
Recuperacion	PasswordReset

El modelo de datos implementa un sistema de permisos granular con las entidades **Rol**, **Permiso** y **RolPermiso**, permitiendo asignaciones flexibles. El modulo de Comercio es el mas extenso, cubriendo el ciclo completo desde compras a proveedores hasta ventas a clientes, incluyendo presupuestos y reservas. El sistema de auditoria registra todas las acciones criticas para trazabilidad completa.

5. Módulos principales y API endpoints

El sistema expone mas de 60 API endpoints organizados por modulo funcional. A continuacion se detallan los principales endpoints y sus metodos HTTP disponibles:

Modulo	Endpoint	Metodos
Productos terminados	/api/productos-terminados	GET, POST
Materias primas	/api/materias-primas	GET, POST
Insumos	/api/insumos	GET, POST
Recetas	/api/recetas	GET, POST
Produccion	/api/produccion	GET, POST
Compras	/api/compras	GET, POST
Ventas	/api/ventas	GET, POST
Pedidos clientes	/api/pedidos-clientes	GET, POST
Presupuestos	/api/presupuestos	GET, POST
Personas	/api/personas	GET, POST
Usuarios	/api/usuarios	GET, POST
Opiniones	/api/opiniones	GET, POST, PATCH
Consultas	/api/consultas	GET, POST, PATCH
Entregas	/api/logistica/entregas	GET, POST
Puntos encuentro	/api/logistica/puntos-encuentro	GET, POST
Reportes stock	/api/reportes/stock	GET
Reportes ventas	/api/reportes/ventas	GET
Reportes compras	/api/reportes/compras	GET
Autenticacion	/api/auth/[...nextauth]	POST
2FA	/api/2fa/activate, verify, disable	POST
Seguridad	/api/seguridad/roles, sesiones, logs-acceso	GET
Notificaciones	/api/notificaciones/*	GET, POST
Subida imagenes	/api/upload/[...slug]	POST

Cada endpoint implementa validacion de permisos mediante los helpers **requireAuth()**, **requireRole()** y **requireAdmin()**. Los endpoints de escritura (POST, PATCH, DELETE) registran automaticamente entradas en la tabla de auditoria. Los endpoints de reportes generan archivos exportables en formatos Excel, PDF y CSV.

6. Variables de entorno (.env)

Las siguientes variables de entorno son necesarias para la configuracion del sistema. Las marcadas como (requerido) son obligatorias para el funcionamiento:

Variable	Descripcion	Ejemplo
DATABASE_URL	URL de conexion Prisma	file:./db/custom.db
DATABASE_AUTH_TOKEN	Token Turso (produccion)	(solo prod)
NEXTAUTH_SECRET	Clave JWT	(requerido)
NEXTAUTH_URL	URL base NextAuth	https://laspastasdeorlando.com.ar
NEXT_PUBLIC_APP_URL	URL publica para emails	https://laspastasdeorlando.com.ar
SMTP_HOST	Servidor SMTP	smtp.gmail.com
SMTP_PORT	Puerto SMTP	587
SMTP_USER	Usuario SMTP	laspastasdeorlando@gmail.com
SMTP_PASS	Contrasena app Gmail	(app password)
BLOB_READ_WRITE_TOKEN	Token Vercel Blob	(solo prod)
ADMIN_EMAIL	Email admin	(requerido)
ADMIN_WHATSAPP	WhatsApp admin	543754419324
TEXTMEBOT_APIKEY	API key TextMeBot	(opcional)

En desarrollo local, se utiliza el archivo **.env.local** para definir estas variables. El archivo **.env.example** del repositorio contiene una plantilla con todas las variables necesarias. La variable **DATABASE_AUTH_TOKEN** solo se requiere en produccion para la conexion con Turso; en desarrollo local se usa SQLite sin token.

7. Despliegue

El sistema se despliega de forma automatica a traves de Vercel, con un flujo CI/CD integrado con GitHub. A continuacion se detallan los aspectos clave del despliegue:

Aspecto	Detalle
Hosting	Vercel (standalone output)
Base de datos	Turso (libSQL) en produccion
Proceso	Push a main -> GitHub -> Vercel deploy automatico
Imagenes	Vercel Blob Storage
Dominio	laspastasdeorlando.com.ar (Caddy reverse proxy)

El proceso de despliegue es completamente automatico: al realizar un push a la rama **main** del repositorio GitHub, Vercel detecta los cambios y ejecuta el build y despliegue. La configuracion de Next.js utiliza **output: 'standalone'** para optimizar el tamaño del bundle. El dominio personalizado laspastasdeorlando.com.ar esta configurado con Caddy como reverse proxy para terminacion SSL.

Flujo de despliegue:

- El desarrollador realiza cambios localmente y hace push a la rama main
- GitHub recibe el push y notifica a Vercel via webhook
- Vercel ejecuta npm run build generando el output standalone
- Vercel despliega la aplicacion en su infraestructura edge
- La aplicacion se conecta a Turso (libSQL) para la base de datos
- Las imagenes subidas se almacenan en Vercel Blob Storage
- El dominio laspastasdeorlando.com.ar apunta a la aplicacion via Caddy

8. Seguridad

El sistema implementa un modelo de seguridad multicapa que incluye autenticacion, autorizacion granular, auditoria completa y proteccion de rutas. A continuacion se detallan los aspectos principales:

8.1 Autenticacion

- NextAuth.js v4 con Credentials provider
- 2FA opcional: TOTP con Speakeasy + codigos de respaldo
- Contraseñas hasheadas con bcryptjs

8.2 Roles y permisos

- Roles definidos: admin, produccion, ventas, lectura
- Permisos granulares: modulo.accion (ej: productos.crear)
- Asignacion flexible via tablas RolPermiso y UsuarioRol

8.3 Sesiones

- JWT con expiracion de 24 horas
- Registro de sesiones activas en tabla SesionActiva
- Cierre de sesion remoto disponible para administradores

8.4 Auditoria

- Registro de todas las acciones: CREATE, UPDATE, DELETE, LOGIN, EXPORT
- Logs de acceso: intentos de login, IP, user agent, resultado
- Trazabilidad completa de cambios en datos criticos

8.5 Proteccion de rutas

- requireAuth(): verifica que el usuario este autenticado
- requireRole(): verifica que el usuario tenga un rol especifico
- requireAdmin(): acceso exclusivo para administradores

Capa	Implementacion
Autenticacion	NextAuth.js v4 + Credentials + 2FA TOTP
Autorizacion	RBAC con permisos granulares (modulo.accion)

Capa	Implementacion
Sesiones	JWT 24hs + registro en SesionActiva
Auditoria	Log de acciones (CREATE, UPDATE, DELETE, LOGIN, EXPORT)
Logs acceso	IP, user agent, resultado de intentos de login
Proteccion rutas	requireAuth(), requireRole(), requireAdmin()

9. Dependencias principales

El proyecto utiliza un ecosistema amplio de dependencias para cubrir todas las funcionalidades del ERP. A continuacion se listan las principales organizadas por categoria:

Categoria	Paquete	Version
Framework	next	^16.1.1
UI	react, react-dom	^19.0.0
Lenguaje	typescript	^5
ORM	@prisma/client, prisma	^6.11.1
Turso	@prisma/adapters-libsql, @libsql/client	^0.17.3
Auth	next-auth	^4.24.11
2FA	speakeasy	^2.0.0
Estilos	tailwindcss	^4
Componentes	25+ @radix-ui/react-*	varios
Iconos	lucide-react	^0.525.0
Animaciones	framer-motion	^12.23.2
Estado	zustand, @tanstack/react-query	^5.x
Tablas	@tanstack/react-table	^8.21.3
Formularios	react-hook-form, zod	^7.x, ^4.x
Mapas	leaflet, react-leaflet	^1.9.4, ^5.0.0
PDF	@react-pdf/renderer, jspdf	^4.5.1, ^4.2.1
Excel	xlsx	^0.18.5
Email	nodemailer	^8.0.10
Seguridad	bcryptjs	^3.0.3
Codigos	jsbarcode, qrcode	^3.12.3, ^1.5.4
Imagenes	sharp, @vercel/blob	^0.34.3, ^2.4.0
Fechas	date-fns	^4.1.0

El proyecto depende de mas de 25 componentes Radix UI que forman la base de la libreria shadcn/ui. La combinacion de React Hook Form + Zod proporciona validacion de formularios

tipada y declarativa. Para la generacion de reportes, se utilizan tres librerias diferentes segun el formato de salida requerido: SheetJS para Excel, @react-pdf/renderer para PDF complejos, y jsPDF para PDF simples.

10. Guia de instalacion local

Para configurar el entorno de desarrollo local, siga los siguientes pasos en orden. Asegurese de tener instalados Node.js (v18+), npm y git antes de comenzar:

Paso 1: Clonar repositorio

```
git clone https://github.com/orlandocandia/pastas-orlando.git
```

Paso 2: Instalar dependencias

```
npm install
```

Paso 3: Configurar variables de entorno

Copiar .env.example a .env.local y completar las variables del punto 6

Paso 4: Ejecutar migracion

```
npx prisma migrate dev
```

Paso 5: Poblar la base de datos (opcional)

```
npm run seed
```

Paso 6: Iniciar servidor de desarrollo

```
npm run dev
```

Paso 7: Acceder a la aplicacion

```
http://localhost:3000
```

Notas adicionales:

- Para desarrollo local se utiliza SQLite, sin necesidad de configurar Turso
- Las variables SMTP son necesarias solo para probar el envio de emails
- TEXTMEBOT_APIKEY es opcional y solo se requiere para notificaciones WhatsApp
- BLOB_READ_WRITE_TOKEN es solo para produccion (Vercel Blob Storage)
- El seed completa la base con datos de ejemplo para todas las tablas

Una vez completados todos los pasos, la aplicacion estara disponible en <http://localhost:3000>. El dashboard de administracion se accede en `/admin/login` con las credenciales configuradas durante el seed o creadas manualmente.